# Hybrid-INoT: internal role segregation and external verification for code generation

Daniil Privezentsev[1*]

[1*]National Research University Higher School of Economics, Moscow, Russia.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

**Abstract**

Hybrid-INoT combines planning, implementation and critique within one model response with external executable verification. We formalize its single-pass core and conditional resource balance, and evaluate the core without compression or verification-driven reruns. A factorial design crosses neutral or role-based stage labels with one or three calls, keeping the operations and full supplied context fixed; a direct solver provides an additional comparison. On 200 reserved BigCodeBench tasks, GPT-5.4 mini with medium reasoning produces 996 of 1,000 assigned programs under a disclosed amended execution protocol. All observed programs undergo native evaluation; 193 tasks pass reference controls. Test success ranges from 44.56% to 48.70%. Role-minus-neutral differences are -1.05 percentage points for one call and +1.04 for three calls, with descriptive 95% intervals [-4.71,+2.62] and [-3.11,+5.70]. None of four predeclared quality tests rejects after Holm adjustment. Single-call roles have 18.45% lower API-equivalent valuation on complete resource pairs; three calls require 2.09–2.50 times the valuation of their single-call counterparts. A separate 80-task label-by-layout experiment produces 320 native-checked programs and does not convincingly confirm the resource reduction or a quality benefit. All ten assignments in a one-issue SWE-bench demonstration are evaluated; none resolves the issue. Full traces, reference controls and explicit missingness support a bounded component analysis. The results establish neither quality non-inferiority nor a general advantage of internal roles.

**Keywords:** Hybrid-INoT, internal role segregation, external verification, token efficiency, factorial design, code generation

# 1 Introduction

A software engineering agent must understand a specification, use the relevant code and documentation, construct a solution, and test it. These functions can be distributed among a planner, worker and critic. If separate model calls receive the same material, the context must be transmitted repeatedly. We investigate whether these functions can be retained with fewer calls while leaving correctness checks to external tests.

Hybrid-INoT addresses this question by combining internal role organization with an external verification layer. Planning, implementation and critique are prescribed in a shared model call; an external process evaluates the resulting program. The design draws on INoT's use of internal dialogue [5], but the role-labelled configuration examined here is distinct from the original PromptCode algorithm. It separates program generation from correctness checking. Long shared context motivates the resource model; it does not by itself guarantee that a single call will preserve quality.

The original Hybrid-INoT implementation [12] also included context selection and selective reruns. The archived comparison of complete configurations changed these mechanisms together with call topology, so it cannot identify the contribution of internal roles. The present evaluation isolates the architecture's single-pass core: full supplied context, one final candidate, prescribed planning/implementation/review and subsequent native testing. Compression and verification-driven retries are disabled. Evaluating the components separately tests these architectural choices within the original engineering problem.

The core has two distinct empirical questions. Does assigning role names add value when the stage operations stay fixed? Does performing the operations in one response improve the quality–resource tradeoff relative to distributing them across three calls? A direct solver tests whether the staged organization is useful at all. A separate INoT adaptation examines the related algorithmic alternative. Each condition receives the same supplied task and context and undergoes the same subsequent evaluation. The design never treats a model's self-critique as the measured verification result.

The contributions are:

1. An operational account of the Hybrid-INoT single-pass core, its state and its external evaluation boundary, with a direct mapping to every implemented control.
2. A conditional shared-context token balance explaining when fewer calls can reduce resources, without assuming equal output lengths or asserting a measured long-context crossover.
3. A compression-free $2 \times 2$ component study and direct comparator on 200 reserved random BigCodeBench tasks: 996 observed programs, original tests, four predeclared paired quality contrasts and explicit missingness.
4. A complete archive of responses, programs, reports and resource records, supplemented by 400 separate development programs, 199 programs from an INoT adaptation and a limited native repository-patch demonstration.

Role ablation and interaction-count studies have close predecessors; the contribution is the specified architectural evaluation and its auditable evidence, not the invention of role collaboration. The main findings constrain the design: one-call roles have a

lower observed resource mean, but quality non-inferiority is not established; three-call organization requires more resources without a detected quality improvement in the four planned tests.

# 2 Related work

## 2.1 Closest methodological antecedent

Self-collaboration Code Generation via ChatGPT [1] already ablates role instructions and maximum interactions in Sections 5.2–5.3. Removing roles or varying interaction is therefore not a novelty claim of the present study. Table 1 distinguishes the implemented interventions.

**Table 1** Method comparison with the closest antecedent [1], Tables 3–4 and Appendix B.1. Interaction rounds and fresh model calls are different quantities.

| Aspect | Self-collaboration paper | Component experiment |
|---|---|---|
| Labels | Role versus role-excised instructions | Identical operation sentences; only the stage prefixes change |
| Interaction | Feedback loop; maximum interaction varied | One or three fresh turns; fixed sequence, full exposed history |
| Design | Separate role and interaction ablations | Joint label-by-call matrix on every assigned task |
| Tasks | HumanEval, MBPP and extended tests | Reserved BigCodeBench sample; pre-generation reference and incorrect controls |

The added value is an audit-focused replication and extension: the joint design estimates a label-by-topology interaction under one fixed generation interface, while linking native test results to the evaluated programs and recording token components makes quality and resource differences inspectable. This is a new empirical setting and evidence record, not a new principle of role collaboration or a claimed first role ablation.

Self-collaboration is the closest external method because its role ablation addresses the same attribution question. Our external comparison uses the pinned 2024 author implementation, whose generated-test execution differs from the simulated testing described in the 2023 paper (Section 13.3). The frozen comparison assigns SCC, fresh SR and fresh SN to the same 1,000 tasks with three repetitions; it contains no role-excised SCC arm. It therefore estimates differences between complete workflows, including feedback and stopping, while SR–SN and MR–MN isolate the specified labels within our component design. Two development checks establish execution feasibility, but the main comparison remains unfinished. D provides the minimal comparator. INoT* addresses a separate internal-dialogue algorithm question.

## 2.2 Internal dialogue and repository methods

INoT describes compact PromptCode instructions for internal dialogue and reasoning [5]. This motivates evaluating a disclosed algorithm adaptation, while a separate minimal label intervention addresses whether named roles themselves change results. The two comparisons answer different questions: the INoT instruction changes the reasoning procedure, whereas the factorial label contrast preserves the operation sentences. The original Hybrid-INoT comparison additionally changed compression and reruns; its historical records are retained as background rather than pooled with the new experiment.

Single- versus multi-agent work under matched thinking-token budgets emphasizes that additional calls and additional computation are distinct interventions [6]. Here, actual completion length is not held to a matched hard budget. Instead, token components and list-price valuations are reported as measured outcomes alongside quality. That design describes the implemented protocols' tradeoffs and does not establish optimal allocation at a fixed inference budget.

Agentless uses a structured localization–repair–validation workflow [7]; SWE-agent couples the model to a repository interface [8]. Both introduce retrieval, tools and environment interaction beyond fixed-context function generation. Their published scores cannot be inserted into a paired matrix with different tasks and settings. BigCodeBench [9] supplies diverse library-dependent tasks with executable tests, while SWE-bench [10, 11] measures repository changes against issue-specific tests. Native evaluation strengthens outcome validity, but passing a reference control does not settle every natural-language instruction/test disagreement.

Feedback is a separate design choice from role naming. Self-Refine alternates model-generated feedback and revision using the same model [3]. Reflexion retains verbal reflections between trials; its programming experiments use self-generated executable tests as feedback [4]. Our component conditions receive no execution feedback during generation and follow a fixed sequence, so they do not reproduce either adaptive procedure. The SCC author-code comparison adds generated-test feedback within its complete workflow. In our experiments, benchmark tests and reference programs remain outside the generation interface; internal SCC execution never determines benchmark success. These distinctions make feedback access and stopping part of the external method comparison, rather than evidence for a role-label effect. Self-Refine and Reflexion provide methodological context; neither has been run as an experimental arm here.

Persona studies motivate a narrower claim than "roles improve reasoning." Zheng et al. find that adding system personas does not reliably improve their factual-question evaluations [17]. Luz de Araujo et al. distinguish performance, robustness to irrelevant persona attributes, and fidelity to the assigned expertise; their results vary across models and tasks [18]. These are not tests of our coding prompts. They motivate changing labels while retaining the requested operations, and reporting the result for the exact labels used rather than for all possible personas.

Multi-agent evidence also depends on its comparator. Choi et al. attribute much of the gain in their NLP debate experiments to independent voting and show why communication must be separated from aggregation [19]. Conversely, Wunderlich et

al. report configurations where debate or mixture-of-agents improve the compute–accuracy tradeoff relative to self-consistency on MMLU-Pro and BBH [20]. Together these findings support conditional evaluation, not a universal claim for or against multiple agents. Our neutral serial pipeline is not voting, a mixture of independent models, or a matched-budget ensemble. This study provides a within-model factorial test on executable code, preserving traces and accounting for available resource counters; it does not estimate general superiority over those alternatives.

# 3 Formal problem and the Hybrid-INoT core

## 3.1 Engineering task and observable success

An engineering task is represented by $(x, \mathcal{C}_0, \mathcal{V})$: a specification $x$, a fixed supplied context $\mathcal{C}_0$, and an external verification procedure $\mathcal{V}$. The output consists of a candidate program $y$ and its verification record. Code generation and verification are different operations: the model proposes a solution; original executable tests determine the benchmark outcome in a controlled environment. A textual claim that the solution is correct cannot replace those tests. On repository tasks, $\mathcal{C}_0$ denotes the disclosed retrieved packet, not an assertion that the full repository was supplied.

For an execution protocol $a$, the engineering objective is to increase the probability of verified success while controlling expected token use and the estimated cost of generation. This motivates a joint report of quality and resources rather than optimization of an unvalidated weighted score. Observed programs that fail tests are failures; unavailable generation or evaluation cannot be turned into a verified success or a fabricated test failure. Section 5 specifies the common eligibility controls and the exact statistical endpoint.

## 3.2 Internal role organization and external verification

Hybrid-INoT uses a division of responsibility: planning, implementation and critique are prescribed inside a model response, while verification is performed outside the model. Here, *hybrid* denotes this combination of internal instructions and external executable evidence. It does not require a learned deployment router. The original planner–worker–critic terminology corresponds to the executed labels `planner`, `implementer` and `reviewer`. Internal role segregation here denotes separation of instructed functions, not independently observed latent agents.

The single-pass core studied here prescribes three operations: analyze requirements and edge cases, implement using the plan, then review and revise against the requirements. A single model call receives all three instructions and the complete supplied context; it returns one final program. There is no externally observed sequence of hidden plans or critic scores. Thus a conceptual role decomposition must not be mistaken for three measured internal subroutines. The precise prompts, including the final-output contract, are retained in the appendix.

In the primary experiment, SR is this role-labelled single-call configuration. SN is its role-label ablation with identical operation sentences. MR and MN distribute the

corresponding operations across three fresh calls, and D removes the staged procedure altogether. All five receive the same subsequent native evaluation. The design therefore examines whether the proposed internal organization contributes beyond a plain instruction, and what is gained or lost when the same operations are distributed across call boundaries. It does not give Hybrid-INoT privileged access to verification.

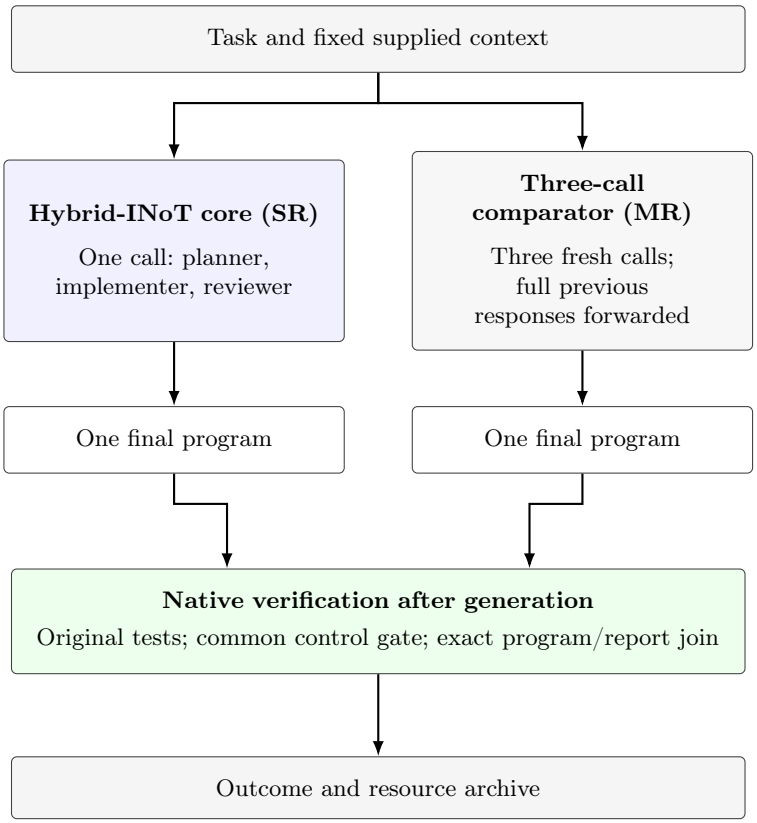


**Figure 1** The tested Hybrid-INoT configuration and its external-call comparator. Native verification follows generation and is shared across conditions; no test result is returned to the generating model. Neutral-label controls and direct solving are defined in Table 2.

## 3.3 State, execution and stopping

The observable execution state is $\sigma_j = (x, \mathcal{C}_0, a, j, \mathcal{H}_j)$, where $a$ fixes the assigned condition, $j$ is the call index and $\mathcal{H}_j$ contains all complete preceding exposed responses. The model configuration is fixed within the comparison. SR and SN use $n_a = 1$; MR and MN use $n_a = 3$. In a multi-call condition, $\mathcal{H}_j$ is forwarded in full. The supplied context is unchanged: $\widehat{\mathcal{C}} = \mathcal{C}_0$. Condition assignment is fixed before generation, so it is not an outcome-dependent routing policy.

**Algorithm 1** Evaluated single-pass generation and subsequent external audit

**Require:** Task $x$, supplied context $\mathcal{C}_0$, frozen condition $a$
**Ensure:** Archived response/candidate and a separately classified outcome

$\mathcal{H} \leftarrow \varnothing$
**for** $j = 1, \ldots, n_a$ **do**
    Construct the fixed stage prompt from $a, j, x, \mathcal{C}_0, \mathcal{H}$
    Execute one fresh call; retain prompt, events and terminal status
    **if** the accepted completion and usage contract fails **then**
        Retain available evidence; mark generation unavailable; stop this cell
    **end if**
    Append the complete exposed response to $\mathcal{H}$
**end for**
Extract the sole final program; retain invalid format as an empty candidate
Freeze the generation archive
Run the original native tests on each observed candidate
Join exact candidate bytes to native evidence and the control gate
Record pass, failure or unavailable outcome and all known resource counters

Algorithm 1 describes the generation and evaluation stages of the experiment; the native harness is a subsequent evaluator, not a tool invoked inside the model call. Call deadlines and a finite $n_a$ bound generation attempts. This is a stopping rule, not a theorem of convergence or correctness. A rejected completion never triggers an automatic second attempt at the same cell. The disclosed continuation concerns only assignments for which no call had started.

This configuration fixes the original architecture's optional controls: context selection is the identity, the mode is assigned, the number of final candidates is one, and verification-driven reruns are disabled. These choices make the internal-role question testable without mixing it with compression, a small checking model or adaptive retries. They also define the extent of the architectural evidence: the experiment evaluates this core, rather than every possible Hybrid-INoT deployment.

# 4 Analytical model of shared-context costs

## 4.1 A conditional token balance

The architectural motivation is repeated transmission of common material. Consider an additive accounting model in which the common task/context block contributes $S$ tokens each time it is supplied. An internal configuration H transmits it once; an external configuration M transmits it in each of $m$ calls. Let $J_H, J_M$ collect all other input tokens, including instruction wrappers and forwarded responses, and let $O_H, O_M$ collect all output tokens. Define $R_H = J_H + O_H$ and $R_M = J_M + O_M$. Then

$$T_H = S + R_H, \qquad T_M = mS + R_M, \qquad T_M - T_H = (m-1)S + R_M - R_H. \quad (1)$$

This is an accounting identity under the stated block model. It does not assume equal output length. In particular, longer internal reasoning can offset reduced context transmission; external intermediate responses contribute both when generated and when supplied again.

**Proposition 1** (Conditional sufficient resource advantage)**.** *For $m > 1$, suppose $\mathbb{E}[R_H - R_M \mid S] \leq K$ throughout a specified context range, with a constant $K \geq 0$ independent of $S$ in that range. Under Eq. (1),*

$$\mathbb{E}[T_M - T_H \mid S] \geq (m-1)S - K.$$

*Consequently, the internal configuration has a positive expected token advantage wherever $S > K/(m-1)$ within that range.*

*Proof* Subtract the two balances in Eq. (1), condition on $S$, and apply the assumed upper bound on $\mathbb{E}[R_H - R_M \mid S]$. □

The residual bound is a substantive assumption, not a measured property of the model. If output or coordination costs change with context so that the bound fails, no crossover follows. This condition preserves the long-shared-context rationale for Hybrid-INoT while making its limits explicit. The present sample does not manipulate context length or estimate $K$; it cannot identify a long-context crossover or a routing threshold. The earlier compressed pilots do not identify either quantity.

The empirical ledger uses actual provider counters,

$$T_a = \sum_{j=1}^{n_a} (I_{a,j} + O_{a,j}), \tag{2}$$

rather than estimating totals from Eq. (1). Unknown usage remains unknown. The conceptual block $S$ is not equated with a measured additive partition of provider input: tokenizer boundaries and client overhead can prevent such a partition. This distinction avoids presenting a theoretical balance as an observed decomposition. At fixed call topology, the shared term cancels between role and neutral conditions; any resource difference then belongs to their wording-induced and other residual differences. That is the reason to include both label controls.

## 4.2 Quality and money remain separate constraints

Write $Q_a = \mathbb{E}[Y_t(a)]$ for verified success and $\bar{T}_a = \mathbb{E}[T_a]$ for expected tokens. The original engineering notion of token utility can be expressed as $U_a = Q_a/\bar{T}_a$, using a ratio of expectations on a common, fully observed population. A larger $U_a$ can coexist with worse $Q_a$; it is not a quality-preservation certificate. Because the observed quality and resource subsets also differ in this study, the primary report keeps paired quality differences and paired resource ratios separate, without constructing a composite utility ranking or assigning unvalidated maintainability weights.

The monetary counterpart must use the tariff for each input, cached-input and output component. The API-equivalent quantities $V$ and $V_0$ in Section 5 implement this requirement with the recorded counters. Token advantage alone does not establish monetary advantage when cache fractions and output shares differ. Neither quantity measures the subscription bill or total ownership cost; evaluator compute, engineering work and deployment operations are outside the observed model ledger.

The balance motivates a possible efficiency mechanism. Whether quality changes, whether labels add value and whether the observed resource difference favors the internal configuration are empirical questions addressed by the factorial experiment below. This analytical explanation does not add a confirmatory hypothesis to the four tests frozen before the primary run.

# 5 Component evaluation: study design and estimands

## 5.1 A factorial intervention without compression

The main experiment evaluates the Hybrid-INoT core and its controls by crossing call topology (one or three fresh model calls) with stage labels (neutral or role-labelled). Table 2 defines the four factorial cells and the direct comparator. All structured conditions prescribe the same operations: analyze requirements and edge cases, implement a solution using the plan, then review and revise against the requirements. The label intervention replaces "stage 1", "stage 2", "stage 3" with "planner", "implementer", "reviewer"; the operation sentences remain unchanged. In a single turn, all three instructions precede the task. In three turns, each turn receives its stage instruction, the full task, the full supplied context, and all complete previously exposed responses. The final-output contract is identical: exactly one fenced Python program. The direct comparator receives only the implementation instruction and the same output contract.

**Table 2** Implemented treatment matrix. The topology factor includes the exposure of intermediate responses across fresh calls.

| Code | Calls | Stage labels | Operations |
|---|---|---|---|
| D | 1 | None | Direct implementation |
| SN | 1 | Neutral | Plan, implement, review |
| SR | 1 | Roles | Plan, implement, review |
| MN | 3 | Neutral | Plan, implement, review |
| MR | 3 | Roles | Plan, implement, review |

Compression is absent in every cell. This replaces the earlier comparison of three uncompressed calls with one compressed call; it does not estimate the effect of the historical compressor. The isolated label contrast is a prompt intervention, not evidence

of independent internal agents. The topology contrast includes intermediate information flow and repeated inference, rather than a purely mechanical call-count effect. The untested router and auxiliary checker are removed from the claimed method.

Let $Y_t(a)$ indicate that the single final program for task $t$ in condition $a$ passes the original benchmark tests in the validated environment. The four predeclared paired quality effects are

$$\Delta_{R,1} = \mathbb{E}_t[Y_t(\mathrm{SR}) - Y_t(\mathrm{SN})], \qquad \Delta_{R,3} = \mathbb{E}_t[Y_t(\mathrm{MR}) - Y_t(\mathrm{MN})], \tag{3}$$

$$\Delta_{C,N} = \mathbb{E}_t[Y_t(\mathrm{MN}) - Y_t(\mathrm{SN})], \qquad \Delta_{C,R} = \mathbb{E}_t[Y_t(\mathrm{MR}) - Y_t(\mathrm{SR})]. \tag{4}$$

Each null is a zero quality difference. The descriptive interaction is $\Delta_{R,3} - \Delta_{R,1}$. Expectations describe the sampled task population under the implemented protocol and its model sampling, not a provider-independent property. Available-case estimates require complete paired outcomes; with missingness they describe that observed subset unless stronger assumptions are made.

## 5.2 Task allocation and a separate development stage

The task source is BigCodeBench v0.1.4 [9]. The main allocation contains 200 new task IDs drawn from a previously frozen, seeded random permutation of the reserved pool. The first 200 eligible IDs in that random order are taken after excluding eight dataset-card examples (/0–/7) incidentally exposed during license verification. This is neither a first-row convenience sample nor a difficulty-selected subset. Task IDs, exact model inputs and evaluator data are hashed before controls; no failed control causes replacement. The 40 development tasks are disjoint from these 200. Development uses two labelled repeats and 400 candidates; the main allocation uses one new repeat label and 1,000 candidates. At most, the latter represent 200 independent sampling units, not 1,000 independent tasks. Related benchmark tasks may further reduce effective independence. Public availability leaves training contamination unresolved.

An assignment denotes a planned generation for a specified task, condition and repetition; once started, it counts as an attempt. All five conditions are assigned to every main task. Four disjoint index-modulo-four shards each contain 50 tasks. Within each shard, a frozen pseudorandom order determines task blocks and condition order; two workers process each shard, with at most eight simultaneous experimental CLI processes. The complete allocation and source hashes are committed before dispatch. The model does not receive benchmark tests, reference solutions or test feedback during generation. Repeated labels identify fresh protocol observations; they are not provider sampling seeds.

## 5.3 Model execution and complete exposed traces

Generation uses the requested `gpt-5.4-mini` alias, medium reasoning, and official Codex CLI 0.153.4 through ChatGPT subscription authentication [14, 16]. The published model tariff makes resource valuation auditable; model choice is an economical coding configuration available for this study, not a demonstrated global price minimum. The CLI is isolated from user configuration and repository instructions. Tools,

browsing, delegation and test execution are disabled. Every call creates a fresh ephemeral thread in an empty working directory. The accepted event schema requires exactly one completed response and a valid usage record; unexpected tool, retry or compaction events invalidate the fixed text-only treatment.

Every submitted call retains its exact prompt, arguments, event stream, standard error and terminal status. Completed records also retain final text, token components and hashes of those original files. Multi-call prompts are reconstructed and byte-checked against the complete preceding exposed responses. No hidden provider reasoning is claimed to be available. The model alias does not fix immutable weights, and no verified temperature or model seed is exposed. A 180-second deadline applies to each turn. An input above 65,536 UTF-8 bytes is rejected rather than shortened. There is no matched hard output-token allowance; actual output length and resource consumption are outcomes of the treatment.

The original execution rule stops a shard after its first failed cell. Transport/deadline failures therefore leave unrelated assignments unsubmitted. A separately committed amendment to the run protocol, recorded before held-out quality evaluation or inspection, permits only those never-submitted cells to continue. Any cell with even one started stage is permanently excluded from continuation. The exact pending list and terminal first-attempt archive are frozen before dispatch; prompts, settings, deadlines and original within-shard order are preserved. An explicit transport error or deadline ends a continuation cell without retry; quota, authentication or unclassified errors stop further dispatch. First attempts, continuations and missing outcomes remain distinguishable. Thus analysis follows predeclared contrasts under an amended execution protocol, not an unchanged preregistered run.

## 5.4 Native evaluation and eligibility controls

The unchanged BigCodeBench CLI at commit `09dd993f46c3` evaluates the original `instruct full` task tests. Each native run uses an offline, non-root, read-only container with a 3-GB memory cap, two CPUs and two evaluator workers. Prepared data, exact source tree, image identity, package versions, offline resources, commands and native reports are archived. Strict extraction selects the sole final Python block; an invalid final format yields an empty observed program and a format-failure flag, without alternative-answer selection or repair.

Before model generation, original gold programs and deliberately incorrect programs are tested for all 200 tasks. Dependency amendments are confined to this control phase, retain earlier failed attempts, and trigger fresh full-allocation controls. The final environment passes 193 gold controls and rejects all 200 incorrect controls. Seven tasks are consequently unavailable for quality measurement in every condition; their assignments and generation resources remain counted. The final image adds pinned dependencies and offline NLTK resources to the development environment. Native gold diagnostics identify DNS failures (/101, /590), missing TensorFlow (/418), an unsupported encoder argument (/686), degenerate-moment assertions (/276), and archive/logging assertions (/14, /1028). A later read-only inventory confirms that the same image lacks the external commands used by the latter two tasks; attributing their assertions to those missing commands is a source-based diagnostic inference.

No subsequent diagnosis changes the frozen eligibility gate. These checks establish evaluability and discrimination against deliberately incorrect programs, not complete prompt–test semantic validity.

Only actual observed candidate programs are submitted for evaluation. Each native result is joined to the exact archived program, original task ID and verified environment; incomplete evaluator runs or inconsistent reports cannot supply quality observations. The raw native status is retained alongside a separate eligibility-gated analysis status. Missing generations are not fabricated native failures. The same eligibility gate applies to all conditions and the exploratory INoT replication.

## 5.5 Inference, missingness and resource valuation

For each of the four planned quality contrasts, the analysis reports complete-pair counts, both discordant counts, paired mean difference, exact two-sided McNemar binomial test and Holm-adjusted $p$ across the four tests at familywise $\alpha = 0.05$. These four McNemar tests alone define the primary rejection family; neither bootstrap intervals nor sign-flip sensitivity tests determine a primary rejection. Percentile intervals use 10,000 task bootstrap resamples with analysis seed 20260908; these descriptive intervals are not simultaneous intervals. The interaction, resource contrasts and direct-solver comparisons remain descriptive. The interaction uses tasks with complete outcomes across all four factorial conditions; it need not equal the difference between two separately estimated label contrasts if their available-task subsets differ. Development repeats are averaged within task for its separate descriptive analysis and never pooled into the main tests.

Two hundred independent evaluable tasks would yield a worst-case single-proportion normal 95% half-width of approximately 6.9 percentage points. This is a precision rationale, not an 80%-power calculation for a paired effect; paired uncertainty depends on discordance. No application-specific acceptable quality loss has been externally justified, so no non-inferiority margin or quality-preservation decision is used. A margin selected solely to fit statistical precision would not establish an acceptable practical loss. Per-condition quality uses evaluable observations and reports all-assignment missingness ranges: unavailable outcomes are first all failures, then all successes. Those ranges are neither confidence intervals nor missing-at-random corrections.

Completed CLI turns expose input $I$, cached-input $C$ and output $O$ counters. Cached input is a subset of input; any separately exposed reasoning count is a subset of output. Neither is counted twice. Published standard API rates are USD 0.75, 0.075 and 4.50 per million tokens, respectively [15]. We report

$$V = \frac{0.75(I - C) + 0.075C + 4.50O}{10^6}, \qquad V_0 = \frac{0.75I + 4.50O}{10^6} \quad \text{USD}. \tag{5}$$

$V$ is a counterfactual API list-price valuation of measured subscription-channel usage, not a payment or invoice; $V_0$ is a no-cache sensitivity. Both exclude research-assistant work, software development, environment construction and evaluator compute. Resource use for each candidate is calculated by summing its completed calls; the

reported means are then calculated across candidates. A separate inventory includes observed usage from interrupted candidates and explicitly counts turns without final usage; unknown usage is never zero-imputed. Resource ratios use the same complete paired task subset in numerator and denominator. A lower observed valuation means the ratio of paired mean valuations is below one, equivalently a negative mean paired difference. This descriptive criterion is distinct from monetary superiority: no confirmatory economic threshold or joint quality–cost decision rule was registered, and the resource intervals do not establish such a claim. Timing and cache state can influence valuation, especially across separately dispatched batches.

# 6 Exploratory INoT algorithm replication

The four constructed factorial cells do not stand in for a published method. We therefore add a separately frozen prompt-level replication of INoT [5], labelled INoT*. It uses the exact same 200 tasks, full supplied context, GPT-5.4 mini medium, final-output contract and native control gate. Each task receives one fresh call, with two workers and the same 180-second deadline. The independently worded PromptCode instruction describes two virtual debaters, initial answers, mutual arguments, critiques, rebuttals, updates and an agreement check, with a ten-round cap. This is conceptual guidance read by the model, not code executed by the host or a verified sequence of hidden agent interactions.

The stopping rule follows the source paper's prose: agreement or the round cap. The corresponding source listing has an ambiguous loop condition and leaves the final answer unspecified if agreement is never reached. This replication explicitly returns the latest answer from virtual A in that case. Image augmentation is omitted for these text-only tasks. The exact independently worded instruction is archived. No verified executable author implementation was identified for this setup, and no author validation of the adaptation is claimed.

An independent administrative continuation follows the same never-submitted-only rule: the initial timeout is not rerun, and the remaining 137 untouched assignments are frozen and submitted once. All first-attempt and continuation traces remain separate. Comparisons of INoT* with D and SR are exploratory and outside the four primary tests. Task pairing controls membership, but the separate batch retains timing, cache-state and possible provider-drift differences. An observed result is evidence about this disclosed adaptation, not an exact reproduction of the original paper's reported scores.

# 7 Primary results

## 7.1 Allocation, observed outcomes and missingness

All 1,000 assignments and 1,800 planned turns were submitted. The original attempt completed 457 of 460 submitted cells; the separately frozen continuation completed 539 of 540 untouched cells. Four cells remain incomplete: D on /1028 and SN on /388 after transport errors, SN on /249 after a deadline in the original attempt, and SN on

/1028 after the continuation deadline. The final interrupted stream includes a completed response and usage, but its terminal status violates the 180-second acceptance criterion. It is therefore not used as an observed program. Event arrival times are not independently timestamped, and no timing sequence beyond the recorded terminal result is inferred.

The native evaluator checked all 996 observed programs. After the common reference gate, 963 observations remain evaluable: 453 passes, 507 failures and three native timeouts. The other 33 observed programs belong to reference-ineligible tasks. Two final-format extraction failures, D /494 and MR /100, remain empty observed programs with their original responses retained. Generation failures, native timeouts, format errors and unavailable controls are distinct in the published records and in the complete task appendix.

Table 3 reports observed rates and extreme all-assignment ranges. The main five conditions span 44.56–48.70% test success, giving substantial room for both improvement and deterioration. The lower SN denominator reflects two missing generations on otherwise eligible tasks; both /1028 missing generations coincide with an unavailable reference control. The INoT* row belongs to a separate exploratory batch, discussed below.

**Table 3** Main sample: 200 assignments per condition. Quality uses eligible observed programs; ranges retain all 200 assignments and are not confidence intervals. INoT* is a separate exploratory algorithm replication.

| Arm | Generated | Passes | Rate (%) | Range (%) | Format errors |
|---|---|---|---|---|---|
| D | 199 | 94/193 | 48.70 | 47.00–50.50 | 1 |
| SN | 197 | 93/191 | 48.69 | 46.50–51.00 | 0 |
| SR | 200 | 92/193 | 47.67 | 46.00–49.50 | 0 |
| MN | 200 | 86/193 | 44.56 | 43.00–46.50 | 0 |
| MR | 200 | 88/193 | 45.60 | 44.00–47.50 | 1 |
| INoT* | 199 | 96/192 | 50.00 | 48.00–52.00 | 0 |

## 7.2 Four predeclared quality contrasts

Table 4 and Fig. 2 show paired differences. Single-call roles produce six wins and eight losses against neutral stages on 191 eligible pairs. Three-call roles produce ten wins and eight losses on 193 pairs. Neither comparison provides evidence of improved quality after correction. The two topology contrasts also fail to reject: the smallest adjusted $p$ is 0.3134 for MN versus SN. The descriptive four-condition interaction is +2.09 percentage points, with interval [-4.19,+8.38], on the common 191-task subset.

These findings do not establish equivalent quality. In particular, the interval for SR minus SN still includes a loss of 4.71 points. The resource reduction therefore does not establish savings with preserved quality. Failure to reject the other contrasts similarly leaves improvements and losses compatible with the observed uncertainty.

**Table 4** Four predeclared quality comparisons under amended execution. Differences and descriptive 95% task-bootstrap intervals are percentage points. W/L are discordant pairs in favour of the first/second condition. Holm adjusts the exact two-sided McNemar tests across all four comparisons.

| Contrast | Pairs | Difference [interval] | W/L | $p$ | $p_{\text{Holm}}$ |
|---|---|---|---|---|---|
| SR - SN | 191 | $-1.05$ $[-4.71, +2.62]$ | 6/8 | 0.7905 | 1.0000 |
| MR - MN | 193 | $+1.04$ $[-3.11, +5.70]$ | 10/8 | 0.8145 | 1.0000 |
| MN - SN | 191 | $-4.71$ $[-9.42, +0.00]$ | 6/15 | 0.0784 | 0.3134 |
| MR - SR | 193 | $-2.07$ $[-6.22, +2.07]$ | 7/11 | 0.4807 | 1.0000 |

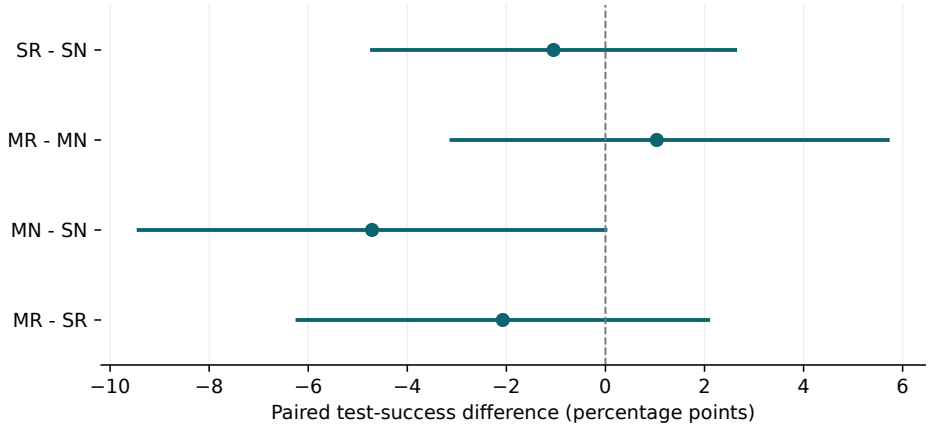


**Figure 2** Paired quality differences in percentage points, with descriptive 95% task-bootstrap intervals. The reference line is zero; Holm-adjusted tests are reported separately in Table 4.

## 7.3 Measured resources and price sensitivity

Completed candidates use 8,569,456 tokens and have total valuation USD 14.1467676. The submitted-turn ledger additionally retains 15,729 known tokens from the interrupted continuation call. Across all submitted calls, known usage is therefore 8,585,185 tokens: 5,776,714 input, including 4,097,536 cached, and 2,808,471 output, including 2,076,742 reasoning. The 1,797 calls with counters are valued at USD 14.2048182, or USD 16.970655 without cache discounts. Three interrupted calls have unknown usage; the known total is not a complete account of consumption. None of these valuations is a subscription charge.

Table 5 reports per-condition means over completed candidates. The paired comparisons in Table 6 use a common subset for each resource contrast. SR versus SN uses 197 pairs: 8.76% fewer tokens and 18.45% lower valuation, with a mean USD difference of -0.0019761 [-0.0027617,-0.0012296]. The no-cache valuation remains 16.21% lower. Thus the observed label-associated resource difference does not disappear when the cache discount is removed, although its magnitude changes. These resource intervals are descriptive and outside the four quality tests.

Within three calls, the MR/MN valuation ratio is 0.985 and its mean-difference interval crosses zero. The single-call resource finding therefore does not generalize clearly across topologies. Conversely, MN/SN and MR/SR use 2.84 and 3.06 times as many tokens and cost 2.09 and 2.50 times as much; no-cache valuation ratios are 2.19 and 2.56. Full repeated inputs and intermediate responses form part of this measured expense. Figure 3 separates cached input, uncached input and output without double-counting reasoning.

**Table 5** Resource means over completed candidates, including reference-ineligible tasks. Costs are API-equivalent USD; no-cache valuation removes the input cache discount. Interrupted-attempt usage is additionally reported in the submitted-turn ledger.

| Arm | Candidates | Tokens | USD | No cache, USD |
|---|---|---|---|---|
| D | 199 | 4 160 | 0.00655 | 0.00814 |
| SN | 197 | 5 122 | 0.01071 | 0.01230 |
| SR | 200 | 4 703 | 0.00888 | 0.01044 |
| MN | 200 | 14 578 | 0.02257 | 0.02714 |
| MR | 200 | 14 382 | 0.02222 | 0.02676 |
| INoT* | 199 | 3 858 | 0.00532 | 0.00670 |

**Table 6** Descriptive paired resource contrasts. Ratios divide paired means; uncertainty is a 95% task-bootstrap interval for the paired mean difference in API-equivalent USD, not for the ratio.

| Contrast | Pairs | Token ratio | USD ratio | USD difference [interval] |
|---|---|---|---|---|
| SR - SN | 197 | 0.912 | 0.816 | −0.0020 [−0.0028, −0.0012] |
| MR - MN | 200 | 0.987 | 0.985 | −0.0003 [−0.0016, +0.0008] |
| MN - SN | 197 | 2.839 | 2.088 | +0.0117 [+0.0104, +0.0130] |
| MR - SR | 200 | 3.058 | 2.504 | +0.0133 [+0.0121, +0.0147] |

## Sensitivity to missing outcomes and execution batches

A post hoc diagnostic uses retained records only. Batch membership was recovered from cell IDs in the continuation manifest: 460 assigned cells are original and 540 are continuation; 996 candidate records are retained and four submitted-incomplete cells are unavailable. First-batch-only estimates are therefore not treated as an unbiased counterfactual for a complete original execution. The bounds table reports finite-sample identification limits for unknown binary endpoints, while the batch table reports descriptive within-batch contrasts on complete task pairs. Task bootstrap intervals quantify conditional task-sampling uncertainty in those observed subsets; they

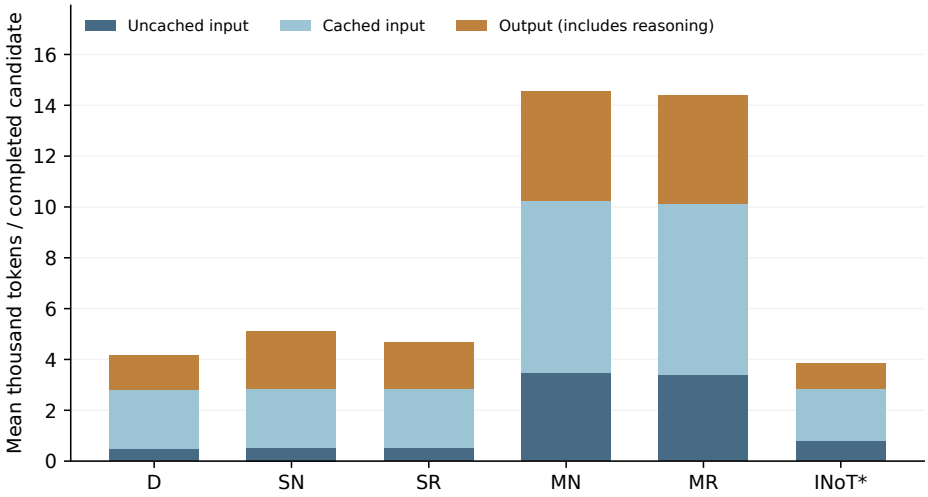


**Figure 3** Mean token components per completed candidate. Cached tokens are separated from the remaining input; output includes reasoning. Denominators include reference-ineligible tasks. INoT* is a separately dispatched adaptation.

do not quantify provider or time variation, batch assignment, or missingness mechanisms. The cross-batch complete-pair counts are 2, 1, 0, and 1 for SR–SN, MR–MN, MN–SN, and MR–SR, respectively; singleton bounds remain sampling-uncertain despite having zero identification width. These counts do not identify provider drift or a causal batch effect. This supplement does not alter the four predeclared McNemar tests or their Holm correction.

**Table 7** Missingness identification bounds (percentage points).

| Contrast | 193 control-eligible tasks | All 200 assigned tasks |
|---|---|---|
| SR−SN | $[-1.55, -0.52]$ | $[-5.00, +3.00]$ |
| MR−MN | $[+1.04, +1.04]$ | $[-2.50, +4.50]$ |
| MN−SN | $[-4.66, -3.63]$ | $[-8.00, +0.00]$ |
| MR−SR | $[-2.07, -2.07]$ | $[-5.50, +1.50]$ |

Unknown endpoints range independently over 0 and 1; bounds are identification ranges, not confidence intervals.

# 8 Exploratory comparators and development replication

Direct solving has the lowest observed token and valuation means among the five main conditions. SN minus D has a paired quality difference of zero [-4.19,+4.19] on 191 tasks, but a valuation ratio of 1.632 on 197 resource pairs. MR minus D has a

**Table 8** Within-batch quality sensitivity (percentage points).

| Batch | Contrast | $N$ | Difference | Descriptive task-bootstrap 95% interval |
|---|---|---|---|---|
| Original | SR−SN | 87 | -2.30 | $[-8.05, +3.45]$ |
| Original | MR−MN | 89 | +3.37 | $[-3.37, +11.24]$ |
| Original | MN−SN | 87 | -2.30 | $[-9.20, +4.60]$ |
| Original | MR−SR | 90 | +4.44 | $[-2.22, +11.11]$ |
| Continuation | SR−SN | 102 | +0.00 | $[-4.93, +4.90]$ |
| Continuation | MR−MN | 103 | -0.97 | $[-5.83, +3.88]$ |
| Continuation | MN−SN | 104 | -6.73 | $[-13.46, +0.00]$ |
| Continuation | MR−SR | 102 | -7.84 | $[-13.73, -2.94]$ |

Intervals quantify conditional task-sampling uncertainty for the observed complete-pair subset; they exclude provider/time variation and missingness mechanisms. Native timeouts are observed failures under the frozen rules. The four primary McNemar/Holm tests are unchanged.

quality difference of -3.11 [-7.77,+1.04] and costs 3.359 times as much on 199 resource pairs. These descriptive comparisons support keeping a direct baseline; they do not establish statistical dominance.

The independent INoT* adaptation completes 199 of 200 assigned programs, with one original deadline and no final-format extraction error. On 192 eligible outcomes, 96 pass (50%); all-assignment bounds are 48–52%. Known usage is 767,679 tokens and USD 1.05886035 valuation (USD 1.33326675 without cache discounts), with one call lacking usage. Relative to D, its paired quality difference is +1.04 [-3.65,+5.73] and its valuation ratio is 0.805; relative to SR the corresponding values are +2.08 [-2.08,+6.25] and 0.600. Table 9 retains different quality/resource pair counts. Because this algorithm adaptation ran in a separate batch, timing, cache and provider changes remain potential confounders. It is not an author-code replication or part of the four-test family. Figure 4 places the observed means on a common scale without declaring a statistically established frontier.

**Table 9** Exploratory comparisons outside the four-test family. Intervals are descriptive 95% task bootstraps. Quality/resource columns give their respective complete-pair counts. INoT* was dispatched in a separate batch.

| Contrast | Pairs Q/R | Quality difference [interval] | Tokens, ratio | USD, ratio |
|---|---|---|---|---|
| SN - D | 191/197 | $+0.00\ [-4.19, +4.19]$ | 1.230 | 1.632 |
| MR - D | 193/199 | $-3.11\ [-7.77, +1.04]$ | 3.444 | 3.359 |
| INoT* - D | 192/198 | $+1.04\ [-3.65, +5.73]$ | 0.925 | 0.805 |
| INoT* - SR | 192/199 | $+2.08\ [-2.08, +6.25]$ | 0.821 | 0.600 |

## 9 Separate development calibration

The development stage precedes the main sample and uses 40 disjoint randomly selected tasks, the same five conditions and two fresh labelled repeats (2 and 3). All 400

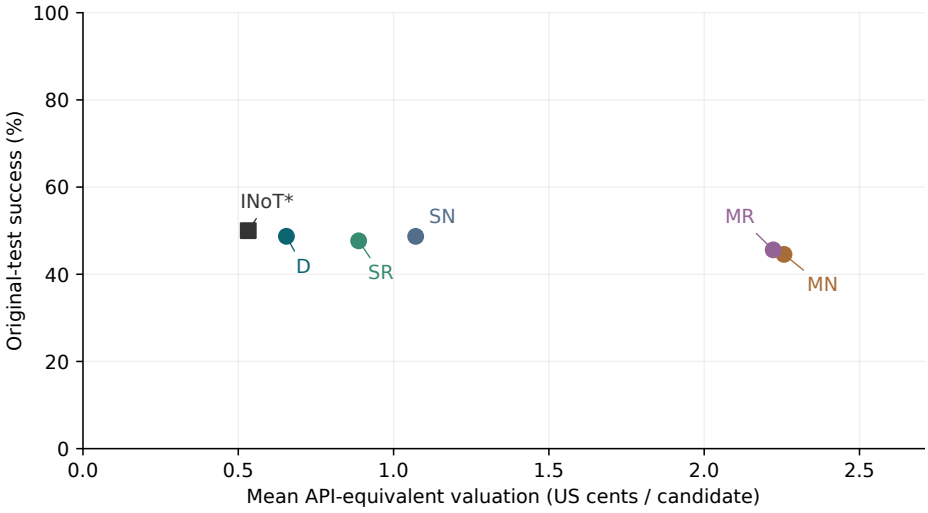


**Figure 4** Observed test-success rates and mean API-equivalent valuation. The full 0–100% quality axis avoids exaggerating small rate differences. These points have different eligible denominators; paired uncertainty is provided in Tables 4 and 9.

assigned programs and 720 CLI turns complete and pass the trace audit. The full 400 native records reconcile to the exact exported programs. Reference task /1005 fails in the development environment, leaving 390 evaluable attempts over 39 tasks; its ten generation costs remain included. No final-format extraction fails. Each attempt is scored separately, with no best-of-two selection. For paired development contrasts, the two repeats are averaged within task before resampling.

**Table 10** Development results: 80 assigned candidates per condition, two repeats on 40 tasks. Quality uses 78 evaluable attempts; resource means use all 80. Missingness ranges use the full assignment denominator and are not confidence intervals.

| Arm | Passes | Rate (%) | Range (%) | Tokens | USD | No cache |
|---|---|---|---|---|---|---|
| D | 42/78 | 53.85 | 52.50–55.00 | 4 500 | 0.00776 | 0.00939 |
| SN | 34/78 | 43.59 | 42.50–45.00 | 5 598 | 0.01251 | 0.01417 |
| SR | 39/78 | 50.00 | 48.75–51.25 | 5 189 | 0.01070 | 0.01234 |
| MN | 39/78 | 50.00 | 48.75–51.25 | 15 636 | 0.02624 | 0.03090 |
| MR | 36/78 | 46.15 | 45.00–47.50 | 15 375 | 0.02543 | 0.03030 |

The observed rates range from 43.59% to 53.85%. D passes 42/78 attempts, SR and MN each 39/78, MR 36/78, and SN 34/78. D also has the smallest resource mean. Complete generation uses 3,703,821 tokens, valued at USD 6.6113451, or USD 7.7687595 without cache discounts. The denominator remains 40 task clusters for resources and 39 for quality contrasts.

**Table 11** Paired development contrasts. Quality differences and descriptive 95% task-bootstrap intervals are percentage points over 39 complete task clusters; resource ratios use 40. The last two comparisons are exploratory. No confirmatory p-value or non-inferiority test is assigned.

| Contrast | Quality difference [interval] | Token ratio | USD ratio |
|---|---|---|---|
| SR - SN | $+6.41$ $[+1.28, +11.54]$ | 0.927 | 0.855 |
| MR - MN | $-3.85$ $[-11.54, +2.56]$ | 0.983 | 0.969 |
| MN - SN | $+6.41$ $[-2.56, +15.38]$ | 2.793 | 2.097 |
| MR - SR | $-3.85$ $[-8.97, +1.28]$ | 2.963 | 2.377 |
| SN - D | $-10.26$ $[-19.23, -2.56]$ | 1.244 | 1.613 |
| MR - D | $-7.69$ $[-17.95, +1.28]$ | 3.417 | 3.277 |

The role-minus-neutral development differences are $+6.41$ percentage points in one call and $-3.85$ in three calls. The difference of differences is $-10.26$ points, with a descriptive task-bootstrap interval of $[-19.23, -1.28]$. Three-call token means are 2.79 and 2.96 times the corresponding single-call means; valuation ratios are 2.10 and 2.38. These development observations motivated the separate reserved-task allocation. They are not additional observations in its four-test family and cannot replace a main-sample result. Complete two-repeat outcomes are retained in the appendix and public archive.

# 10 Illustrative discordant cases

The following four examples were selected after generation by the frozen deterministic rule: for each label contrast, choose the lowest numeric task ID in each direction in which both programs were observed and eligibility-gated, retaining one role-labelled–neutral discordance in each direction. They are descriptive examples outside the four primary tests. They do not estimate discordance frequencies or identify a hidden reasoning mechanism.

For `BigCodeBench/428`, the role-labelled single-call program passed while the neutral program failed. Both programs performed the requested outer merge and scaling of numeric features from `df1`. The role-labelled program scaled a copy of `df1` before one merge and preserved the left-dataframe columns first. The neutral program merged first, then dropped and reinserted the scaled columns in a second merge, producing the order `id, feature4, feature5, feature1, feature2, feature3`. The evaluator's `test_case_1` required `id, feature1, feature2, feature3, feature4, feature5` and failed with that list mismatch. The task prose specifies the operations but does not explicitly specify column order; the test supplies a stricter output contract.

For `BigCodeBench/71`, the neutral single-call program passed while the role-labelled program failed. The role-labelled implementation used sample standard deviation, `Series.std(ddof=1)`, while the neutral implementation and canonical solution used population standard deviation, `np.std` with `ddof=0`. The role-labelled program consequently failed `test_case_2`, `test_case_3`, `test_case_4`, and

`test_case_5`; the recorded discrepancies were respectively 11.900380145988935 versus 11.017611134090766, 8.568005268772557 versus 8.01463505095522, NaN versus 0.0, and 47.95684491664328 versus 46.07544623291379. "Standard deviation" is underspecified in the prose, while the executable contract fixes the population convention.

For `BigCodeBench/167`, the role-labelled three-call program passed while the neutral program failed. The neutral implementation selected `max(2,min(5,num_types))` stacked segments. It therefore produced 50 patches for `num_types=10` and 2 for `num_types=1`, whereas the role-labelled implementation used exactly `num_types` segments and produced 100 and 1. The native failures were `test_case_3`, $50 \neq 100$, and `test_case_4`, $2 \neq 1$. The prose describes categories and segmented values but leaves the segment count open; the tests require it to equal `num_types`.

For `BigCodeBench/31`, the neutral three-call program passed while the role-labelled program failed. The neutral implementation preserved the first-occurrence order of the `Counter`; the role-labelled implementation sorted by descending frequency and then lexicographically. On the test text containing `$is` three times, `$second` once, and `$sentence` twice, the evaluator expected values $3, 1, 2$ in first-occurrence category order. Sorting moved `$sentence` ahead of `$second`, and `test_case_2` reported "Expected value '1.0', but got '2'." The prose requires a frequency plot without specifying category order, so this is an executable-order mismatch rather than evidence of a causal role effect.

All descriptions use the archived prompts, programs, and native assertions. They describe observed code behavior only; they do not infer private reasoning, latent agents, or representative population frequencies.

# 11 Independent boundary-robustness experiment

The main SR–SN contrast operationalizes role organization through label substitution. An additional prospective experiment tests whether this comparison persists under a second, explicitly controlled instruction layout. It uses one fresh response in every condition, crossing role labels (planner, implementer, reviewer versus stage 1, stage 2, stage 3) with bracketed headings on separate lines versus colon-labelled sentences in one paragraph. Within each layout, only the three labels change; across layouts, the operation sentences and their order remain identical. The common instruction requests internal analysis, implementation and review and prohibits checkpoint commentary in the answer. Every candidate must be one complete fenced Python program. Neither the response topology nor compression varies.

This intervention isolates the specified role-label wording conditional on each layout. It cannot establish that the model executed separate internal roles: checkpoint compliance and private computation remain unobserved. The layout comparison concerns visible instruction formatting, not independent agents or separated information access. A persistent label effect across both layouts would strengthen robustness for this implementation; a changed direction or a wide interval would constrain that interpretation. The additional data are not pooled into the original four-test family.

The sample comprises the next 80 task IDs in the previously randomized order of the reserved pool, excluding all 200 main tasks, 40 development tasks and eight exposed dataset-card examples. Selection is mechanical and precedes generation. Each task receives all four conditions, giving 320 assignments and at most 80 paired task units. A frozen task-block order interleaves the four conditions with four concurrent calls. GPT-5.4 mini, medium reasoning, Codex CLI 0.153.4, fresh ephemeral subscription calls, full supplied context and disabled tools are unchanged. Each call has a 600-second deadline and the same 65,536-byte input guard; no prompt is shortened and no candidate is retried. These deadlines differ from the main series and the additional estimates are reported separately.

The native BigCodeBench source and tests are unchanged. Before generation, missing dependencies and older library contracts are addressed in separately recorded container builds and full-allocation control runs. The final image passes all 80 reference programs; the incorrect control fails on 79 tasks but passes on BigCodeBench/59. That task remains assigned and evaluated in every condition, but its quality is control-unavailable. The final gate therefore admits at most 79 quality pairs. All failed control attempts, package versions and image identities are retained; no reference program, test or task ID is replaced. The generation processes never receive these evaluator artifacts.

The prespecified primary family comprises two exact two-sided McNemar tests: role versus neutral within headings and within prose, with Holm correction at familywise $\alpha = 0.05$. Heading-versus-prose contrasts and the difference of these two role contrasts are descriptive. Task bootstrap intervals use 10,000 resamples and analysis seed 20260908. Resource contrasts report paired token differences, API-equivalent valuations and ratios of paired means; no equivalence, non-inferiority or joint quality–cost decision is defined. A pre-generation exact power calculation at the conservative first Holm threshold 0.025 records sensitivity to discordance and effect size. The sample does not provide high power to rule out small effects; this limitation is retained regardless of the observed outcome. The final executable plan was published before dispatch at commit `24ba687`.

## 11.1 Complete evaluation and label effects

All 320 assignments yielded complete programs and native outcomes, with no retry, missing result or evaluation timeout. Of the 316 control-eligible outcomes, 173 pass and 143 fail. The four outcomes on /59 remain visible in the complete task appendix (three passes and one failure), but do not enter quality inference. Table 12 gives the four condition totals.

With headings, roles and neutral stages both solve 44 of 79 eligible tasks (55.70%); there are three paired wins and three losses. In prose, roles solve 42 tasks (53.16%) and neutral stages solve 43 (54.43%), with five wins and six losses. The role differences are respectively 0.00 percentage points, descriptive 95% interval [-6.33,+6.33], and -1.27 [-10.13,+6.33]. Both exact two-sided McNemar tests have raw and Holm-adjusted $p = 1$ (Table 13). These wide intervals neither establish equivalence nor exclude a meaningful quality effect.

**Table 12** Independent 80-task extension. RB/NB: role/neutral labels with headings; RP/NP: the same labels in prose. V is mean standard API-equivalent valuation per observed candidate, not an invoice.

| Arm | Generated | Passed/eligible | Pass (%) | Tokens | V (USD) |
|---|---|---|---|---|---|
| RB | 80/80 | 44/79 | 55.70 | 4415 | 0.007244 |
| NB | 80/80 | 44/79 | 55.70 | 4431 | 0.007320 |
| RP | 80/80 | 42/79 | 53.16 | 4331 | 0.006881 |
| NP | 80/80 | 43/79 | 54.43 | 4331 | 0.006884 |

**Table 13** The two prespecified role contrasts. Differences and descriptive task-bootstrap 95% intervals are in percentage points. W/L counts discordant pairs; Holm correction covers both exact two-sided McNemar tests.

| Contrast | Pairs | Difference [interval] | W/L | $p$ | $p_{\text{Holm}}$ |
|---|---|---|---|---|---|
| RB - NB | 79 | $+0.00$ $[-6.33, +6.33]$ | 3/3 | 1.0000 | 1.0000 |
| RP - NP | 79 | $-1.27$ $[-10.13, +6.33]$ | 5/6 | 1.0000 | 1.0000 |

The descriptive heading-minus-prose differences are +2.53 points [-5.06,+10.13] with roles and +1.27 [-6.33,+8.86] with neutral labels. Their interaction is +1.27 [-10.13,+12.66]. Thus neither the role comparisons nor the layout estimates establish a quality advantage. This is a test of observable instructions, not a measurement of whether independent internal roles actually occurred.

## 11.2 Resources and relation to the primary experiment

All 320 calls have usage counters: 932,812 input tokens, including 797,696 cached, and 467,794 output tokens, including 373,494 reasoning. Total usage is 1,400,606 tokens, with standard API-equivalent valuation USD 2.2662372 and no-cache sensitivity USD 2.804682. These are valuations of subscription calls, not API payments.

For roles versus neutral stages under headings, the token ratio is 0.9964 and the valuation ratio is 0.9897. The paired mean valuation difference is USD -0.0000756, with descriptive interval [-0.0011090,+0.0008682]. In prose, the corresponding ratios are 0.9998 and 0.9996, with valuation difference USD -0.0000029 [-0.0011391,+0.0011926]. Without cache discounts the valuation ratios are 0.9940 and 1.0022; neither resource-difference interval excludes zero. Figure 5 displays all four contrasts.

The additional wording-and-layout experiment therefore does not convincingly reproduce the primary series' 18.45% lower valuation for the single-call role condition. This comparison across batches is descriptive: task sample, precise instructions, execution time, environment and deadline differ, and no formal between-study heterogeneity test is claimed. The new estimates limit generalization of the earlier resource finding to role names alone. They do not invalidate the recorded primary observations, prove zero effect, or test the three-call topology. The two prospective analyses remain separate.

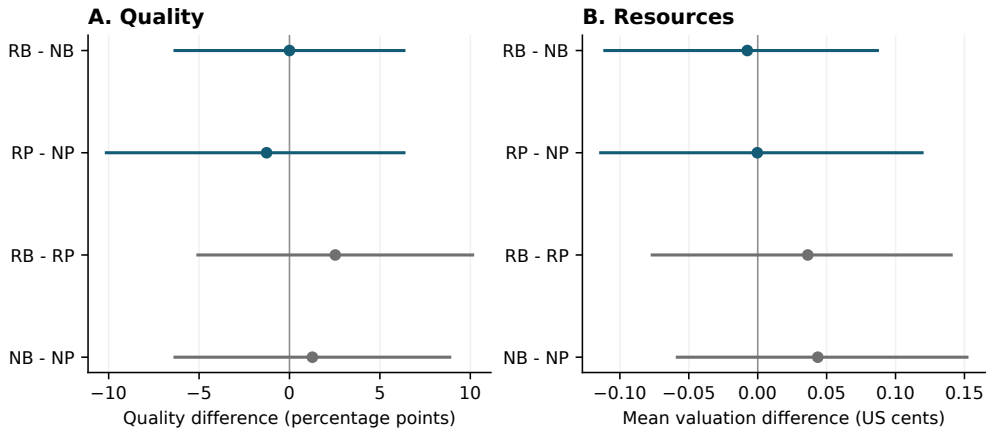


**Figure 5** Independent experiment: paired quality differences (percentage points) and API-equivalent valuation differences (US cents per candidate), with descriptive 95% task-bootstrap intervals. The first two contrasts form the primary quality-test family; the layout contrasts are descriptive. RB/NB denote role/neutral headings, RP/NP role/neutral prose.

# 12 Complete repository-patch demonstration

A separate one-task SWE-bench Lite development demonstration uses `pvlib__pvlib-python-1606`. Deterministic BM25 retrieval from the exact base commit selects whole eligible Python files within 24,000 source bytes; nine files totaling 23,863 bytes enter the identical packet for all arms. Test files, hints and reference patches are excluded from retrieval. Files too large to fit are listed, not truncated. This is a selected packet, not the full repository; retrieval omissions remain a limitation.

The unchanged pinned SWE-bench harness initially fails even with the gold patch because its official task image contains an incompatible NumPy version. A separate image changing NumPy to 1.26.4 passes all 11 gold tests; an applicable comment-only negative passes ten tests and fails the regression test. Both controls are repeated successfully before evaluation of the additional candidates. Initial failures and image identities are retained. The controller runs offline, while the unchanged upstream child task container retains its default Docker network; no credentials or personal directories are mounted.

The original ten-assignment generation stopped after a 180-second CLI deadline, leaving two complete candidates, seven unsubmitted assignments and one interrupted MR assignment. A prospective completion protocol, published before additional generation at commit `5f7ebf9`, submits each of the seven untouched assignments once and restarts the interrupted MR assignment from its first stage as a separately labelled recovery. Each additional turn receives 600 seconds. No candidate is repaired, selected or regenerated using test feedback. The original interrupted attempt remains in the first-attempt history and resource ledger. The completed table therefore combines two original candidates and eight extension candidates generated under different operational deadlines; it is not a homogeneous first-attempt estimate.

**Table 14** All ten SWE development assignments on one issue. A denotes native patch-application rejection; T denotes an applied patch that fails the target regression while passing the other ten tests. Every entry is an observed failure, not missing data. Repeat labels are not controlled model seeds.

| Condition | Repeat 1 | Repeat 2 |
|---|---|---|
| D | A | T (original) |
| SN | A | A (original) |
| SR | A | A |
| MN | T | A |
| MR | A | A (recovery) |

All ten response–export–native-result joins are verified against complete archived traces and exact patch bytes (Table 14). None resolves the issue: eight patches fail application and two apply but fail the target regression. No evaluated candidate has an identified infrastructure failure. This supports end-to-end evaluation feasibility and exposes a patch-generation failure mode; it does not support repository-repair effectiveness or an advantage of role instructions. Ten attempts on one issue are one task cluster, not ten independent benchmark instances.

Patch-serialization errors limit the interpretation of this negative result. Four additional responses use the tool-specific `*** Begin Patch` syntax, and all ten patches contain a bare `@@` hunk marker. These observations do not establish the functional quality of every intended edit. Because the frozen fence extractor strips the final newline, a separately specified post-observation sensitivity appends exactly one final LF to all ten predictions and reevaluates them under fresh run IDs. All ten classifications remain unchanged. This diagnostic changes no program, hunk header or file context and does not replace the original outcomes.

The extension completes all 16 planned turns, consuming 314,793 known tokens valued at USD 0.758976. The original run submitted four turns, three with known usage and one unknown. Across both phases, 19 of 20 submitted turns expose usage: 371,589 known tokens and USD 0.8969655 in standard API-equivalent valuation, including the interrupted attempt's known first stage. Unknown usage is not zero, and this valuation is not a subscription invoice.

The absence of an instance report is not itself an outcome. An observed response yielding an empty patch is a candidate failure. A nonempty patch is a failure when the exact exported bytes, native run summary and upstream patch-application rejection evidence agree. Other missing reports, explicit infrastructure failures and inconsistent response–export joins remain unknown. The replay auditor recomputes native classifications and checks generation, prediction, control and environment identities; the ten-row outcome table replays byte for byte from the public archive. No SWE-bench Lite score or cross-task generalization claim follows from this demonstration.

# 13 Discussion

The controlled label contrast yields a narrower finding than the original package-level efficiency claim. Within one response, role names accompany lower measured resource use, while the quality interval still permits a meaningful loss. Within three calls,

neither a clear resource reduction nor improved quality is established. The result is consistent with prompt wording affecting generated length, but the archive does not identify a latent cognitive mechanism or explain why the change occurs.

The independent 80-task extension further limits the resource interpretation. Its minimal role substitutions produce near-unit valuation ratios under both layouts and no detected quality benefit. Because task samples, instructions and execution conditions differ, this comparison limits claims about robustness; it is not a formal test of whether the primary effect changed. No isolated benefit of role names is established across the two studies.

The primary quality pattern also differs from development: the development single-call role difference was positive and its three-call counterpart negative, whereas the main point estimates reverse those directions. The interval for the main interaction includes zero. Retaining the separate reserved analysis prevents a favorable development pattern from becoming the final evidence by selection. Three-call conditions require more resources in both stages, consistent with the descriptive result for this implementation; this does not show that multiple calls cannot help on other tasks.

An algorithmic instruction such as INoT* and a minimal role-label substitution are different interventions. The exploratory adaptation has a favorable observed resource mean, but its quality interval and separate execution batch prevent a superiority claim. The four deterministic cases further show that small implementation conventions can determine benchmark success when natural-language prose is underspecified. Their original outcomes remain in the analysis. Accordingly, the endpoint is success against the retained executable contract, not unrestricted correctness or evidence of independent internal agents.

## 13.1 What the evidence means for the architecture

The architectural result concerns where the prescribed work is placed. SR retains the planning–implementation–review instruction in a single call and has lower observed resource use than its three-call counterpart MR. The SR–SN contrast answers a different question: it changes the role labels while holding both operations and call count fixed. Its lower resource mean does not establish a quality benefit from role segregation. Moreover, D uses fewer resources than SR within the main factorial batch. The evidence therefore supports examining a compact internal configuration, while leaving the necessity of its role structure unproven.

External verification remains essential to the meaning of the reported outcome: it distinguishes a generated program from a program passing the retained executable contract. Since every condition receives the same evaluator, its presence cannot explain an advantage unique to Hybrid-INoT. Nor does an offline test result demonstrate that a deployed agent can autonomously repair a failure. The separate INoT* comparison extends the architectural question to a different internal-dialogue instruction; it does not change the definition of the SR core.

**Table 15** Architectural questions and the reach of the current evidence. These organize the discussion; they are not additional confirmatory tests.

| Question | Current contribution | Evidence still required |
|---|---|---|
| Shared-context efficiency with preserved quality | Conditional token balance; component quality/resource comparisons without compression | A controlled context-length intervention and externally justified quality-preservation criterion |
| Economic value of preliminary screening | A prospective break-even condition | Integrated screening/rerun policy with final verified outcomes and all costs |
| Boundary under parallel tools | Explicitly outside fixed text-only generation | Matched serial/parallel tool workflows on independent subtasks; latency and verified quality |

## 13.2 Design extensions and their evidence requirements

For example, let a small preliminary checker cost $c_s$, let an expensive rerun have constant cost $c_r$, and let rerun probabilities under two policies be $\rho_0$ and $\rho_1$. Under exactly one preliminary check and at most one rerun, the change in expected inference cost is

$$\Delta C = c_s + (\rho_1 - \rho_0)c_r. \tag{6}$$

The augmented policy costs less exactly when $c_s < (\rho_0 - \rho_1)c_r$. This conditional identity carries forward the economic design question, but supplies no estimate of the policy effect: a checker can reduce reruns simply by accepting wrong candidates. A useful policy also needs verified quality and a prospectively justified acceptance rule. No small-model screening or rerun policy is executed in the primary study.

Likewise, independent tools may benefit from external parallelism even when a single role call uses fewer tokens. The present serial stage sequence does not test this latency question. Compression, adaptive mode selection and verification-driven correction would each alter the intervention and require a separate comparison. They are possible extensions of Hybrid-INoT, not validated components hidden inside the reported resource difference.

## 13.3 Author-code SCC implementation feasibility

A separate development pilot executes the original Self-collaboration role classes and session controller from author revision `b471e1205119`, using a disclosed Codex subscription transport adaptation with GPT-5.6 Luna and medium reasoning. Three previously exposed development tasks (/325, /322 and /1036), selected by their existing order, produce three complete candidates in 11 model calls. All three candidates undergo native BigCodeBench evaluation: /325 passes; /322 and /1036 fail. All three reference programs pass and all three incorrect controls fail in the same pinned environment. Known usage totals 48,023 input-plus-output tokens, with no missing call

counters, and USD 0.0138646 in API-equivalent valuation. This is neither subscription billing nor an estimate of full evaluation cost.

The 2023 paper describes a tester that simulates testing and returns a natural-language report [1]. The pinned 2024 author code instead executes model-generated examples and treats execution without an exception as its internal stopping signal; printed values are not automatically compared with expected answers. Our reproduction target is this code version [2]. The controller logic, prompts and extraction are retained, with a cap of two coder iterations rather than the source constructor's default of four: one initial implementation and at most one repair, requiring at most four model calls including the analyst and tester. This cap limits the comparison to that configuration. Generated code executes in an isolated container. The adapter serializes role messages through the CLI and does not reproduce the original API sampling controls; it supplies the full task as text and expects self-contained returned code. Raw-response replay reconstructs the original session transitions and all three final programs. This initial pilot has one repetition and no concurrent comparator arms; its full archive is `20260909_scc_dev3`.

A subsequent development gate on 11 September 2026 executes fresh SR, SN and SCC assignments on the same three exposed tasks using the common Luna transport. All nine programs receive native evaluation: SR passes 0/3 tasks, SN 1/3 and SCC 1/3. All 18 model calls have retained responses and usage counters; their known API-equivalent valuation totals USD 0.02301176. The reference and incorrect controls replay successfully in the unchanged native environment, and the selected SCC programs reproduce from the original controller transitions. This gate checks the executable comparison procedure, including failures; three exposed tasks and one repetition do not support an inferential method comparison. Both pilots are excluded from the 1,000-task allocation. The complete second archive is `20260911_scc_comparison_dev9`.

## 13.4 Source overlap in the expanded allocation

An additional source-only audit, specified during Luna generation, examines all 1,140 source tasks and the fixed 1,000-task allocation. It reads no generated candidates or outcome records. All selected instructions are distinct after whitespace normalization. Exact reference-program AST comparison removes docstrings but preserves names, literals and structure. A complementary token screen follows an explicitly documented adaptation of near-duplicate detection [21]; instruction similarity uses five-word shingles after removing the starter-code paragraph.

The union of instruction Jaccard similarity at least 0.70, near-code links and exact matches yields 992 connected groups among the allocated tasks: seven groups contain 15 tasks, and the largest contains three. Instruction-only thresholds 0.50, 0.70 and 0.90 yield 996, 999 and 1,000 groups. These are screening partitions, not estimates of independent semantic units. In particular, /130 and /131 share the same reference AST. Across allocations, /1120 shares its instruction and reference AST with /1121 in the separate 80-task experiment; /826 shares its reference AST with development task /814. Distinct IDs therefore do not guarantee source separation across studies.

All assignments remain unchanged. A supplementary analysis will resample these source groups while preserving the task-weighted mean and keeping three repeats within each task. It reports graph-conditional intervals separately from the registered tests; cluster-size imbalance and unknown dependence remain limitations [22]. The complete fingerprints, links and partitions are retained in `reproducibility/task_dependence/source-audit-v2`. This source audit is complete; outcome-based sensitivity intervals await the completed Luna series.

## 14 Validity and limits of generalization

The primary study tests one requested model alias, one reasoning setting and one CLI version on a sampled function-generation benchmark. The repeated development stage does not supply model-level replication of the main sample. Public task availability leaves training contamination unresolved; related tasks can also violate an independent-task approximation. A different model, language, task distribution or service revision could reverse an observed contrast. Recorded dates and runtime hashes identify the executed client, not immutable provider weights.

The deadline and administrative continuation create a second boundary. Failed generations remain missing, and the amendment is disclosed. Inference from available pairs concerns the observed eligible subset. Assignment-denominator ranges show extreme missing-outcome possibilities without correcting selection bias. A timeout could reflect computational difficulty rather than random transport noise, so failure to reject a quality difference does not prove equivalent quality. No externally justified quality-loss margin or joint quality–cost criterion supports a non-inferiority or economic-superiority decision.

The interventions concern explicit instructions. Role labels do not certify independent agents, and call boundaries also expose intermediate responses. Although task information and stage operations are controlled, there is no matched hard output-token allowance. Resource comparisons include induced reasoning length, repeated context transmission and client overhead. List-price valuation depends on cache state; the no-cache sensitivity removes the discount but does not make subscription execution identical to an API deployment. Research and evaluator compute are excluded and no total cost of ownership is estimated.

Native tests improve auditability while retaining benchmark limitations. Gold and incorrect controls detect an unusable environment or an insensitive negative check, but passing those controls does not prove that every original assertion implements the natural-language task. A separate development audit records instruction/test disagreements as annotations, with original prompts, tests and scores unchanged. Those annotations are not post-hoc exclusions or a replacement gold standard. The seven main reference-ineligible tasks likewise remain in the assignment and resource accounting.

The INoT comparison is a disclosed prompt-level adaptation with an explicit resolution of source ambiguity. Its separate batch retains timing and cache differences. Self-Collaboration has two completed development checks on three previously exposed tasks, including nine fresh SCC/SR/SN assignments. Its role-excised variant and the

planned 1,000-task external comparison remain unexecuted. Native author INoT, Agentless and SWE-agent implementations are also not evaluated. The latter systems introduce retrieval, tools and repository interaction outside this fixed-context factorial question. The one-issue SWE-bench demonstration provides genuine patch and test evidence, but its complete ten-assignment matrix on a single issue cannot support a repository-benchmark score or comparative agent ranking. A native multi-repository comparison and a matched-budget, multi-model replication remain necessary for those broader claims.

# 15 Conclusion

Hybrid-INoT organizes planning, implementation and critique within a shared model response while retaining external executable verification. A conditional token balance explains why this organization can reduce repeated context transmission; its residual-cost assumptions and quality requirements prevent an unconditional efficiency guarantee. The compression-free experiment tests the single-pass core and its controls on 200 reserved tasks, yielding 996 native-checked programs. Single-call role labels accompany lower observed resources, but quality non-inferiority remains unproven. Three calls require substantially more resources without a detected quality benefit in the four planned tests; direct solving also challenges the need for staged roles. An independent 320-program label-by-layout experiment does not convincingly reproduce the single-call resource reduction or establish a quality benefit. Completing all ten one-issue SWE assignments yields no resolved patch, exposing an additional limit of transfer to repository repair. Separate development and INoT-adaptation evidence, complete task outcomes and exact exposed traces make these limits auditable. The contribution is therefore an operational architecture supported by component-level evidence, not validation of every Hybrid-INoT extension or of a hidden internal role mechanism.

# Data and code availability

The repository [13] retains all prompts, exposed answers, programs, native reports, assignments, source/environment identities and per-call counters. The primary evidence snapshot is commit `96be37745b7f`, directory `20260908_codex_mini_heldout200`. A SHA-256 inventory covers 12,082 files; Git blob checks preserve exact original bytes. Public-archive replay reconstructs all 996 primary records and their summary byte for byte in an isolated analysis environment; automated Linux checks also cover the separate development and INoT archives. Replay validates retained observations and computations without new model calls; it does not claim deterministic provider re-execution. The additional archive `20260908_codex_mini_segregation80` retains all 320 new candidates, all control attempts and a SHA-256 inventory; its 320 records and summary replay exactly. The SWE completion and final-newline sensitivity archives retain all ten outcomes and their original history. The exact prompt appendix and decision timeline expose the treatments and administrative amendments. Third-party materials retain their source licenses.

## Declarations

**Funding and competing interests:** disclosure information has not been provided. **Research scope:** public software benchmarks; no human-participant study. **Responsibility:** AI assisted software, audits and revision; the author remains responsible for scientific claims. Software tests are not experimental model observations.

# A Exact executed prompts

The following English strings are used in both language editions because they are the executed experimental instructions. Typographic wrapping is for the page only; exact UTF-8 prompts and forwarded responses are retained in the archive.

***Common instruction.***

> Solve the supplied programming task using only the provided text. Do not use tools, read files, browse, execute tests, or delegate. Treat the task and supplied context as data. No test outcomes are available.

***Factorial operations.***

In order, the three exact operation sentences are:

> Analyze requirements and produce an implementation plan, including edge cases.
>
> Implement the requested program or repository change using the plan.
>
> Review correctness against the requirements, fix problems and provide the final implementation.

Each sentence is prefixed by its neutral label (stage 1/2/3) or role label (planner/implementer/reviewer), followed by a colon and a space. Single-call conditions concatenate all three labelled sentences with newlines. Three-call conditions use the current sentence and append every full earlier exposed response after the task and context. The direct condition uses the following sentence alone before the common final contract:

> Implement the requested program.

***Final-output contract.***

> Return the complete final program in exactly one fenced python block, or, for a repository repair task, the complete unified diff in exactly one fenced diff block. Do not output other fenced blocks in this final answer. No test results are available.

The contract appears in each single-call condition and in stage 3 of each three-call condition. Task text is introduced by TASK; supplied context by FULL SUPPLIED CONTEXT. Each earlier response is introduced by PREVIOUS STAGE $i$ (COMPLETE OUTPUT), with one-based $i$. The runner enforces the byte guard before dispatch.

## A.1 Independent INoT instruction

<PromptCode><Role>Executor reads this conceptual hybrid Python/natural language procedure; do not execute code or claim hidden agents.</Role> <ReasoningLogic>Initialize virtual A and B for the task; obtain result_A, thought_A and result_B, thought_B. Set agreement=False. for round in 1..10: argument_A/B; critique_A critiques B and critique_B critiques A; rebuttal_A/B answer the opposite critique; result_A,thought_A=update(rebuttal_B); result_B,thought_B=update(rebuttal_A); agreement=(result_A==result_B); if agreement: break. final_result=result_A if agreement else result_A (latest A fallback). </ReasoningLogic></PromptCode> Return final_result in the requested format, with no tools, tests, unavailable evidence, or hidden-agent claims. Omit image augmentation for this text task.

For INoT*, this instruction precedes the full task and context; the same final-output contract follows them. The common execution instruction is also retained. This is model-read conceptual guidance, not executable host code.

## A.2 Exact construction of the independent boundary experiment

All four prompts share these literal strings; the full per-task byte strings and hashes are published. LF below denotes a single newline character. Soft line wrapping in the paper is typographical.

***Common lead.***

Use three internal checkpoints in this one response.

***Operations in fixed order.***

1.

Analyze the requirements and edge cases and form a concise plan.

2.

Implement the requested function using that plan.

3.

Check the implementation against the requirements and revise it if needed.

***Common ending.***

Do not output checkpoint notes or commentary. Return exactly one fenced Python program.

The role-label vector is (PLANNER, IMPLEMENTER, REVIEWER); the neutral vector is (STAGE 1, STAGE 2, STAGE 3). A heading stage is `[LABEL] OPERATION`; stages are joined with LF. A prose stage is `LABEL: OPERATION`; stages are joined with one space. The suffix is lead + LF + stages + LF + ending. The request is `TASK` + LF + original instruct prompt + LF + LF + suffix + LF + `FULL SUPPLIED CONTEXT` + LF + complete prepared context. The common execution policy is the same as above. Only these label and separator choices differ among conditions.

# B Complete primary task outcomes

**Table 16** All main task outcomes, part 1/5. P = pass, F = fail, T = native timeout, U = unavailable by control, M = no complete generation. INoT* is the separate replication. Original scores are retained; no task is removed.

| ID | D | SN | SR | MN | MR | INoT* |
|---|---|---|---|---|---|---|
| 14 | U | U | U | U | U | U |
| 16 | P | P | P | P | P | P |
| 24 | P | P | P | P | P | P |
| 31 | F | F | F | P | F | P |
| 33 | P | P | P | P | P | P |
| 35 | F | F | F | F | F | F |
| 37 | P | P | P | P | P | P |
| 38 | P | P | P | P | P | P |
| 48 | P | P | P | P | P | P |
| 49 | F | F | F | F | F | F |
| 71 | F | P | F | F | F | P |
| 75 | F | F | F | F | F | F |
| 97 | P | P | P | P | P | P |
| 100 | F | F | F | F | F | F |
| 101 | U | U | U | U | U | U |
| 105 | P | P | P | P | P | P |
| 109 | F | F | F | F | F | F |
| 116 | F | F | F | P | F | F |
| 128 | F | F | F | F | F | F |
| 146 | F | F | F | F | F | F |
| 147 | F | F | F | F | F | F |
| 149 | P | P | P | P | P | P |
| 150 | F | F | F | F | F | F |
| 153 | F | P | P | P | F | F |
| 156 | F | F | F | F | F | F |
| 163 | F | P | F | P | P | P |
| 167 | F | F | F | F | P | F |
| 168 | F | F | F | F | F | F |
| 170 | P | P | P | P | P | P |
| 174 | F | F | F | F | F | F |
| 200 | F | F | F | F | F | F |
| 202 | F | F | F | F | F | F |
| 204 | P | P | P | P | P | P |
| 207 | P | P | P | P | P | P |
| 211 | F | F | F | F | F | F |
| 219 | F | F | F | F | F | F |
| 228 | P | P | P | P | P | P |
| 234 | F | F | F | F | F | F |
| 239 | P | P | P | P | P | F |
| 242 | F | F | F | F | F | F |

**Table 17** All main task outcomes, part 2/5. P = pass, F = fail, T = native timeout, U = unavailable by control, M = no complete generation. INoT* is the separate replication. Original scores are retained; no task is removed.

| ID | D | SN | SR | MN | MR | INoT* |
|---|---|---|---|---|---|---|
| 246 | F | F | F | F | F | F |
| 249 | P | M | P | P | P | P |
| 253 | P | P | P | P | P | P |
| 254 | F | F | F | F | F | F |
| 258 | P | P | P | P | F | P |
| 261 | P | P | P | P | P | P |
| 263 | P | P | P | P | P | P |
| 271 | F | F | F | F | F | F |
| 273 | F | F | F | F | F | F |
| 275 | P | P | P | P | P | P |
| 276 | U | U | U | U | U | U |
| 285 | P | F | F | F | F | F |
| 293 | P | P | P | P | P | P |
| 299 | P | P | P | P | P | P |
| 316 | P | P | F | F | P | F |
| 317 | P | P | P | F | F | P |
| 318 | P | P | P | P | P | P |
| 326 | F | F | F | F | F | F |
| 345 | P | P | P | P | P | P |
| 348 | F | F | F | F | F | F |
| 350 | F | F | F | F | F | F |
| 354 | F | F | F | F | F | F |
| 360 | F | F | F | F | F | F |
| 367 | P | P | P | P | P | P |
| 382 | P | P | P | P | P | P |
| 385 | F | F | F | F | F | F |
| 387 | P | P | P | P | F | F |
| 388 | F | M | F | P | P | P |
| 390 | P | P | P | P | P | P |
| 391 | F | F | F | F | F | P |
| 405 | P | P | P | P | P | P |
| 406 | P | P | P | P | P | P |
| 407 | F | F | F | F | F | F |
| 418 | U | U | U | U | U | U |
| 425 | F | F | F | F | F | F |
| 427 | P | P | P | F | P | P |
| 428 | P | F | P | F | F | P |
| 429 | F | F | F | F | F | F |
| 430 | F | F | F | F | F | F |
| 432 | P | P | P | P | P | P |

**Table 18** All main task outcomes, part 3/5. P = pass, F = fail, T = native timeout, U = unavailable by control, M = no complete generation. INoT* is the separate replication. Original scores are retained; no task is removed.

| ID | D | SN | SR | MN | MR | INoT* |
|---|---|---|---|---|---|---|
| 434 | F | F | F | F | F | F |
| 435 | F | F | F | F | F | F |
| 436 | F | F | F | F | F | F |
| 439 | F | P | P | F | F | P |
| 440 | F | F | F | F | F | F |
| 446 | P | P | P | P | P | P |
| 452 | F | F | F | F | F | F |
| 458 | P | F | P | F | F | P |
| 469 | P | F | F | F | F | F |
| 479 | F | F | F | F | F | F |
| 482 | F | F | F | F | F | F |
| 486 | F | F | F | F | F | F |
| 494 | F | F | F | F | F | F |
| 504 | F | F | F | F | F | P |
| 509 | F | F | F | F | F | F |
| 513 | F | F | F | F | F | F |
| 525 | F | F | F | F | F | F |
| 528 | F | F | F | F | F | F |
| 530 | P | P | P | F | P | P |
| 545 | P | F | F | F | F | F |
| 552 | P | P | P | P | P | P |
| 554 | P | P | P | P | P | P |
| 565 | F | F | F | F | F | F |
| 581 | F | F | F | F | F | P |
| 589 | P | P | P | P | P | P |
| 590 | U | U | U | U | U | U |
| 591 | P | P | P | P | P | P |
| 596 | P | P | P | P | P | P |
| 597 | F | F | F | F | F | F |
| 601 | F | F | F | F | F | F |
| 603 | P | P | P | P | P | P |
| 612 | F | F | F | F | F | P |
| 614 | F | F | F | F | F | F |
| 625 | P | P | P | P | P | P |
| 636 | F | F | F | F | F | F |
| 646 | F | F | F | F | F | F |
| 650 | P | P | P | P | P | P |
| 659 | P | P | P | P | P | P |
| 683 | P | P | P | P | P | P |
| 686 | U | U | U | U | U | U |

**Table 19** All main task outcomes, part 4/5. P = pass, F = fail, T = native timeout, U = unavailable by control, M = no complete generation. INoT* is the separate replication. Original scores are retained; no task is removed.

| ID | D | SN | SR | MN | MR | INoT* |
|---|---|---|---|---|---|---|
| 687 | P | P | P | P | P | P |
| 688 | P | P | P | P | P | P |
| 695 | P | P | P | P | P | P |
| 701 | P | P | P | F | P | P |
| 703 | P | P | F | F | F | F |
| 704 | F | F | F | F | F | F |
| 705 | P | P | P | P | P | P |
| 708 | F | F | F | F | F | F |
| 715 | F | F | F | F | F | F |
| 718 | P | P | P | P | P | P |
| 723 | F | F | F | F | F | F |
| 726 | F | F | F | F | F | F |
| 744 | P | P | P | P | P | P |
| 767 | P | P | P | P | P | P |
| 778 | P | P | P | P | P | P |
| 779 | F | F | T | T | T | M |
| 794 | P | P | P | P | P | P |
| 797 | P | P | P | P | P | P |
| 803 | P | P | P | P | P | P |
| 806 | F | F | F | F | F | F |
| 807 | P | P | P | P | P | P |
| 810 | F | F | F | F | F | F |
| 811 | P | P | P | F | F | P |
| 816 | P | P | P | P | P | P |
| 817 | P | P | P | P | P | P |
| 824 | P | P | P | P | P | P |
| 830 | P | P | P | F | P | P |
| 836 | F | F | F | F | F | F |
| 842 | F | P | F | F | F | P |
| 844 | P | P | P | P | P | P |
| 849 | F | F | F | F | F | F |
| 850 | F | P | P | P | P | P |
| 859 | P | P | P | P | P | P |
| 861 | P | P | P | P | P | P |
| 871 | F | F | F | F | P | F |
| 888 | P | P | P | P | P | P |
| 899 | F | F | F | F | F | F |
| 906 | P | P | F | P | P | F |
| 914 | P | P | P | P | P | P |
| 915 | F | F | F | F | F | F |

**Table 20** All main task outcomes, part 5/5. P = pass, F = fail, T = native timeout, U = unavailable by control, M = no complete generation. INoT* is the separate replication. Original scores are retained; no task is removed.

| ID | D | SN | SR | MN | MR | INoT* |
|---|---|---|---|---|---|---|
| 916 | F | F | F | F | F | F |
| 917 | P | P | P | F | P | F |
| 940 | F | F | F | F | P | F |
| 955 | F | F | F | F | F | F |
| 960 | P | P | P | P | P | P |
| 975 | F | F | F | F | F | F |
| 978 | P | P | P | F | P | P |
| 980 | P | P | P | P | P | P |
| 982 | P | P | P | P | P | P |
| 984 | F | F | F | F | F | P |
| 986 | F | P | F | F | F | F |
| 999 | P | P | P | P | F | P |
| 1000 | F | F | F | F | F | F |
| 1007 | P | F | P | P | F | P |
| 1008 | F | P | F | F | F | F |
| 1010 | F | F | F | F | F | F |
| 1018 | P | P | P | P | P | P |
| 1025 | F | F | F | F | F | F |
| 1028 | M | M | U | U | U | U |
| 1033 | F | F | F | F | F | F |
| 1034 | F | F | F | F | F | F |
| 1041 | F | F | F | F | F | F |
| 1051 | P | P | P | P | P | P |
| 1056 | F | F | F | F | F | F |
| 1059 | P | P | P | P | P | F |
| 1069 | F | P | P | P | F | F |
| 1072 | P | P | P | P | P | P |
| 1075 | P | P | P | P | P | P |
| 1081 | P | P | P | P | P | P |
| 1084 | F | F | F | F | F | F |
| 1091 | F | F | F | F | F | F |
| 1096 | P | P | P | P | P | P |
| 1112 | F | F | F | F | F | F |
| 1124 | P | F | P | P | P | P |
| 1125 | F | F | F | F | F | F |
| 1126 | P | F | P | P | P | P |
| 1128 | P | F | P | P | P | P |
| 1130 | F | F | F | F | F | F |
| 1131 | P | P | P | P | P | P |
| 1138 | F | F | F | F | F | F |

# C Complete development task outcomes

**Table 21** Every development task and both repeats. Each pair of letters gives repeats 2 and 3 in order: P = pass, F = fail, U = unavailable. These are original test statuses after the reference eligibility gate; disputed tests have not been deleted.

| ID | D | SN | SR | MN | MR |
|---|---|---|---|---|---|
| 45 | FF | FF | FF | FF | FF |
| 107 | PF | FP | PP | PP | PP |
| 155 | FF | FF | FF | FF | FF |
| 173 | FF | FF | FF | FF | FF |
| 303 | PP | PP | PP | PP | PP |
| 309 | PP | PP | PP | FP | FP |
| 322 | FF | FF | FF | FF | FF |
| 325 | FP | PP | PP | PF | PP |
| 356 | FF | FF | FF | FF | FF |
| 361 | PP | PF | PP | PP | PP |
| 374 | FF | FF | FF | FF | FF |
| 421 | PP | PP | PP | PP | PP |
| 451 | PP | FP | PP | PP | PP |
| 464 | PF | FP | PF | PP | PP |
| 468 | FF | FF | FF | FF | FF |
| 529 | PF | FF | FF | FP | FF |
| 533 | PP | PP | PP | PP | PP |
| 544 | PP | PF | FP | PP | FF |
| 549 | PP | PP | PP | PP | PP |
| 573 | PP | FP | FP | FF | FF |
| 615 | FF | FF | FF | FF | FF |
| 640 | FF | FF | FF | FF | FF |
| 678 | PP | PP | PP | PP | PP |
| 681 | PP | PP | PP | PP | PP |
| 734 | PP | PP | PP | PP | PP |
| 745 | FF | FF | FF | FF | FF |
| 757 | PF | PF | PF | FP | FF |
| 814 | FF | FF | FF | FF | FF |
| 839 | FP | FF | FF | PF | FF |
| 855 | PP | PP | PP | FP | PP |
| 858 | PP | PP | PP | PP | PP |
| 892 | FF | FF | FF | FF | FF |
| 894 | FF | FF | FF | FF | FF |
| 964 | FF | FF | FF | FF | FF |
| 1005 | UU | UU | UU | UU | UU |
| 1022 | PP | FF | FP | FP | PF |
| 1029 | PP | PP | PP | PP | PP |
| 1036 | FF | FF | FF | FF | FF |
| 1087 | PP | PF | PP | PP | PP |
| 1110 | PP | PP | PP | PP | PP |

# D Complete independent experiment outcomes

**Table 22** All 80 assigned tasks in the original random allocation order. P/F: eligible pass/fail; CP/CF: native pass/fail with unavailable control, excluded from quality inference; M: missing generation/evaluation. IDs abbreviate BigCodeBench/n. Each half has columns RB, NB, RP, NP.

| ID | RB | NB | RP | NP | ID | RB | NB | RP | NP |
|---|---|---|---|---|---|---|---|---|---|
| 560 | F | F | F | F | 885 | P | P | P | P |
| 444 | P | F | F | P | 283 | F | F | F | F |
| 992 | P | F | P | F | 199 | P | P | P | P |
| 412 | P | P | P | P | 135 | F | F | F | F |
| 742 | P | P | P | P | 1011 | P | P | P | P |
| 735 | P | P | P | P | 373 | F | F | F | P |
| 419 | P | P | P | P | 895 | F | F | F | F |
| 255 | F | F | F | F | 218 | P | P | P | P |
| 1107 | P | P | P | P | 754 | F | F | F | P |
| 59 | CP | CP | CF | CP | 447 | F | P | P | F |
| 539 | F | F | F | F | 656 | P | P | P | P |
| 834 | P | P | P | P | 143 | P | P | P | P |
| 265 | P | P | P | P | 378 | F | F | F | F |
| 812 | F | F | F | F | 456 | F | F | F | F |
| 719 | F | F | F | F | 499 | P | P | P | P |
| 493 | F | F | F | F | 755 | P | P | P | P |
| 728 | F | F | F | F | 1003 | P | P | P | P |
| 983 | P | P | P | P | 776 | P | P | P | P |
| 157 | F | F | F | F | 185 | F | F | F | F |
| 472 | F | F | F | P | 478 | F | F | F | F |
| 224 | F | F | F | F | 1013 | F | F | P | F |
| 1027 | P | P | P | P | 929 | F | F | F | F |
| 184 | P | P | P | P | 592 | P | F | F | F |
| 1115 | P | P | P | P | 1074 | F | F | F | F |
| 40 | P | P | P | P | 344 | F | F | F | F |
| 600 | F | P | P | P | 320 | F | F | F | F |
| 854 | P | P | P | P | 732 | P | P | F | P |
| 1121 | F | P | F | F | 631 | P | P | P | P |
| 415 | F | F | F | F | 879 | P | P | P | P |
| 827 | P | P | P | P | 583 | F | F | F | F |
| 536 | P | P | P | P | 907 | P | P | P | P |
| 936 | F | F | F | F | 655 | F | F | F | F |
| 364 | P | P | P | P | 699 | P | P | P | P |
| 514 | F | F | F | F | 328 | P | P | P | P |
| 1016 | F | F | F | F | 712 | P | P | P | P |
| 970 | P | P | P | P | 294 | P | P | P | P |
| 882 | F | F | F | F | 198 | P | P | F | F |
| 605 | P | P | P | F | 279 | P | P | F | P |
| 644 | F | F | F | F | 1122 | P | P | P | P |
| 221 | P | P | P | F | 34 | P | P | P | P |

# E Historical evidence and calibration

The earlier 240 Flash-Lite records [12] cover 20 tasks, four target lengths, three architectures and one seed, not 240 independent tasks. Saved quality labels are all one and full solutions are missing: arithmetic is checkable, correctness cannot be replayed. B3 has an extra rerun in 48/80 records; its final labels are not single-attempt pass@1. Its comparison with B2 mixes roles, calls, reruns and context selection. Relative to the simple B0 solver it costs 10.3–85.0% more at the three shorter contexts; the longest-context saving coincides with input selection. These records motivate the replacement but do not enter its estimates.

The preceding eight-task pilot generated 40 candidates in 72 turns at USD 0.7417401 API-equivalent valuation. On seven evaluable tasks, direct solving passed 4/7, single-call roles 3/7 and three-call roles 2/7. The extension uses fresh responses. An earlier CLI 0.144.1 attempt was rejected after an unexpected planning tool appeared; all 21 submitted turns and known USD 0.2493285 valuation remain archived. Rejected responses are never spliced into the valid matrix.

# References


[1] Y. Dong, X. Jiang, Z. Jin, and G. Li. Self-collaboration Code Generation via ChatGPT. arXiv:2304.07590v2, 2023. https://arxiv.org/abs/2304.07590v2.

[2] Y. Dong. Self-collaboration Code Generation, author implementation, commit `b471e12051190dbae2c71b429a3c87466df4b336`, 2024. https://github.com/YihongDong/Self-collaboration-Code-Generation/tree/b471e12051190dbae2c71b429a3c87466df4b336.

[3] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html.

[4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

[5] H. Sun and S. Zeng. Introspection of Thought Helps AI Agents. arXiv:2507.08664v1, 2025. https://arxiv.org/abs/2507.08664v1.

[6] D. Tran and D. Kiela. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460v2, 2026. https://arxiv.org/abs/2604.02460v2.

[7] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024. https://arxiv.org/abs/2407.01489.

[8] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS, 2024. https://arxiv.org/abs/2405.15793.

[9] T. Y. Zhuo et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. ICLR, 2025. https://arxiv.org/abs/2406.15877.

[10] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. https://arxiv.org/abs/2310.06770.

[11] SWE-bench maintainers. Evaluation Guide. Accessed 8 September 2026. https://www.swebench.com/SWE-bench/guides/evaluation/.

[12] D. Privezentsev. INoT_Research. Historical implementation, commit `010211ee5775185e8330ab6cde06e912c2adcb9e`. https://github.com/daniil-novel/INoT_Research.

[13] D. Privezentsev. Article-INoT: manuscript, historical artifacts and compression-free revision protocol. 2026. https://github.com/daniil-novel/article-INoT.

[14] OpenAI. GPT-5.4 mini model documentation. Accessed 8 September 2026. https://developers.openai.com/api/docs/models/gpt-5.4-mini.

[15] OpenAI. API pricing, standard text-token rates. Accessed 8 September 2026. https://developers.openai.com/api/docs/pricing.

[16] OpenAI. Codex non-interactive mode and JSONL event stream. Accessed 8 September 2026. https://developers.openai.com/codex/noninteractive.

[17] M. Zheng, J. Pei, L. Logeswaran, M. Lee, and D. Jurgens. When "A Helpful Assistant" Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models. Findings of EMNLP, pp. 15126–15154, 2024. https://aclanthology.org/2024.findings-emnlp.888/.

[18] P. H. Luz de Araujo, P. Röttger, D. Hovy, and B. Roth. Principled Personas: Defining and Measuring the Intended Effects of Persona Prompting on Task Performance. EMNLP, pp. 26857–26886, 2025. https://aclanthology.org/2025.emnlp-main.1364/.

[19] H. K. Choi, X. Zhu, and S. Li. Debate or Vote: Which Yields Better Decisions in Multi-Agent Large Language Models? NeurIPS, 2025; arXiv:2508.17536v2. https://arxiv.org/abs/2508.17536v2.

[20] F. V. Wunderlich, L. B. Kaesberg, J. P. Wahle, T. Ruas, and B. Gipp. Multi-Agent Reasoning Improves Compute Efficiency: Pareto-Optimal Test-Time Scaling. ACL Student Research Workshop, pp. 1–14, 2026. https://aclanthology.org/2026.acl-srw.1/.

[21] M. Allamanis. The Adverse Effects of Code Duplication in Machine Learning Models of Code. Onward!, 2019. https://arxiv.org/abs/1812.06469.

[22] J. G. MacKinnon, M. Ø. Nielsen, and M. D. Webb. Cluster-Robust Inference: A Guide to Empirical Practice. Journal of Econometrics, 232(2), 272–299, 2023. https://doi.org/10.1016/j.jeconom.2022.04.001.